```
java Machinetrace ex3.out 4 > ex3trace.txt
```

Exercise 8.2 Compile and run the micro-C example programs you wrote in Exercise 7.2, and check that they produce the right result. It is rather cumbersome to fill an array with values by hand in micro-C, so the function `squares` from that exercise is very handy.

Exercise 8.3 This abstract syntax for preincrement `++e` and predecrement `--e` was introduced in Exercise 7.4:

```
type expr =
    ...
  | PreInc of access   (* C/C++/Java/C#  ++i  or  ++a[e]  *)
  | PreDec of access   (* C/C++/Java/C#  --i  or  --a[e]  *)
```

Modify the compiler (function `cExpr`) to generate code for `PreInc(acc)` and `PreDec(acc)`. To parse micro-C source programs containing these expressions, you also need to modify the lexer and parser.

It is tempting to expand `++e` to the assignment expression `e = e+1`, but that would evaluate `e` twice, which is wrong. Namely, `e` may itself have a side effect, as in `++arr[++i]`.

Hence `e` should be computed only once. For instance, `++i` should compile to something like this: `<code to compute address of i>`, `DUP`, `LDI`, `CSTI 1`, `ADD`, `STI`, where the address of `i` is computed once and then duplicated.

Write a program to check that this works. If you are brave, try it on expressions of the form `++arr[++i]` and check that `i` and the elements of `arr` have the correct values afterwards.

Exercise 8.4 Compile `ex8.c` and study the symbolic bytecode to see why it is so much slower than the handwritten 20 million iterations loop in `prog1`.

Compile `ex13.c` and study the symbolic bytecode to see how loops and conditionals interact; describe what you see.

In a later chapter we shall see an improved micro-C compiler that generates fewer extraneous labels and jumps.

Exercise 8.5 Extend the micro-C language, the abstract syntax, the lexer, the parser, and the compiler to implement conditional expressions of the form `(e1 ? e2 : e3)`.

The compilation of `e1 ? e2 : e3` should produce code that evaluates `e2` only if `e1` is true and evaluates `e3` only if `e1` is false. The compilation scheme should be the same as for the conditional statement `if (e) stmt1 else stmt2`, but expression `e2` or expression `e3` must leave its value on the stack top if evaluated, so that the entire expression `e1 ? e2 : e3` leaves its value on the stack top.

Exercise 8.6 Extend the lexer, parser, abstract syntax and compiler to implement `switch` statements such as this one:

```
switch (month) {
  case 1:
    { days = 31; }
  case 2:
    { days = 28; if (y%4==0) days = 29; }
  case 3:
    { days = 31; }
}
```

Unlike in C, there should be no fall-through from one `case` to the next: after the last statement of a `case`, the code should jump to the end of the `switch` statement. The parenthesis after `switch` must contain an expression. The value after a `case` must be an integer constant, and a case must be followed by a statement block. A `switch` with *n* cases can be compiled using *n* labels, the last of which is at the very end of the `switch`. For simplicity, do not implement the `break` statement or the `default` branch.

Exercise 8.7 (Would be convenient) Write a disassembler that can display a machine code program in a more readable way. You can write it in Java, using a variant of the method `insname` from `Machine.java`.

Exercise 8.8 Write more micro-C programs; compile and disassemble them. Write a program that contains the following definitions of `void` functions:

- A function `linsearch(int x, int len, int a[], int *res)` that searches for `x` in `a[0..len-1]`. It should use pointer `res` to return the least `i` for which `a[i] == x` if one exists, and return -1 if no `a[i]` equals `x`.
- A function `binsearch(int x, int n, int a[], int *res)` that searches for `x` in a sorted array `a[0..n-1]` using binary search. It should use pointer `res` to return the least `i` for which `a[i] == x` if one exists, and return -1 if no `a[i]` equals `x`.
- A function `swap(int *x, int *y)` that swaps the values of `*x` and `*y`.
- A function `sort(int n, int a[])` that sorts the array `a[0..n-1]` using insertion sort. (Or use selection sort and the auxiliary function `swap` developed above).

Exercise 8.9 Extend the language and compiler to accept initialized declarations such as

```
int i = j + 32;
```

Doing this for local variables (inside functions) should not be too hard. For global ones it requires more changes.

precise formalization of the Java Virtual Machine. A more comprehensive effort that also relates the JVM and Java source code, is by Stärk, Schmid, and Börger [27].

The Microsoft Common Language Infrastructure is described by Gough [13], Lidin [16], and Stutz [26]. Microsoft's CLI specifications and implementations have been standardized by Ecma International since 2003 [10].

Kotzmann et al. describe Sun/Oracle's client JVM [15], and Suganuma et al. have a series of papers [28] describing IBM's JVM; in addition, whitepapers at company websites describe these and other implementations of JVM and CLI. A recent trend in the academic literature on just-in-time compilation is the design of so-called tracing JITs, see for instance Bebenita et al. [3].

One can argue that abstract machines, such as the 1936 Turing machine [30], predate the construction of actual stored-program computers in hardware in the mid-1940es. However, Turing's machine, with its very local control and unbounded sequential storage tapes, is quite far removed from practical computer hardware designs.

A particularly beautiful early theoretical development that is also more "realistic" is found in Corrado Böhm's 1951 PhD dissertation. In just 46 pages the dissertation develops an abstract machine, a small programming language, and a compiler from that language to the abstract machine, with the compiler written in the language itself. The original dissertation is in French but there is a translation into English [7].

9.9 Exercises

The main goal of these exercises is to improve understanding of the mainstream virtual machines such as the Java Virtual Machine and the .NET Common Language Infrastructure, including their intermediate code, metadata, and garbage collectors.

Download and unpack `virtual.zip` which contains the programs needed in the exercises below.

Exercise 9.1 Consider the following C# method from file `Selsort.cs`:

```
public static void SelectionSort(int[] arr) {
  for (int i = 0; i < arr.Length; i++) {
    int least = i;
    for (int j = i+1; j < arr.Length; j++)
      if (arr[j] < arr[least])
        least = j;
    int tmp = arr[i]; arr[i] = arr[least]; arr[least] = tmp;
  }
}
```

(i) From a Visual Studio Command Prompt, compile it using Microsoft's C# compiler with the optimize flag (`/o`), then disassemble it, saving the output to file `Selsort.il`:

```
csc /o Selsort.cs
ildasm /text Selsort.exe > Selsort.il
```

Open `Selsort.il` in a text editor, find method `SelectionSort` and its body (bytecode), and delete everything else. Now try to understand the purpose of each bytecode instruction. Write comments to the right of the instructions (or between them) as you discover their purpose. Also describe which local variables in the bytecode (local 0, 1, …) correspond to which variables in the source code.

To see the precise description of a .NET Common Language Infrastructure bytecode instruction such as `ldc.i4.0`, consult the Ecma-335 standard [10], find Partition III (PDF pages 324-471 in the December 2010 version) of that document, and search for `ldc`.

(ii) Now do the same with the corresponding Java method in file `Selsort.java`. Compile it, then disassemble the Selsort class:

```
javac Selsort.java
javap -verbose -c Selsort > Selsort.jvmbytecode
```

Then investigate and comment `Selsort.jvmbytecode` as suggested above. For the precise description of JVM bytecode instructions, see [17, Chap. 6].

Hand in the two edited bytecode files with your comments.

Exercise 9.2 This exercise investigates the garbage collection impact in Microsoft .NET of using repeated string concatenation to create a long string. This exercise also requires a Visual Studio Command Prompt.

(i) Compile the C# program `StringConcatSpeed.cs` and run it with `count` in the program set to 30,000:

```
csc /o StringConcatSpeed.cs
StringConcatSpeed
(and press enter to see next result)
```

You will probably observe that the first computation (using a StringBuilder) is tremendously fast compared to the second one (repeated string concatenation), although they compute exactly the same result. The reason is that the latter allocates a lot of temporary strings, each one slightly larger than the previous one, and copies all the characters from the old string to the new one.

(ii) In this part, try to use the Windows Performance Monitor to observe the .NET garbage collector's behavior when running StringConcatSpeed.

- In the Visual Studio Command Prompt, start `perfmon`.
- In the `perfmon` window, remove the default active performance counters (shown in the list below the display) by clicking the "X" button above the display three times.
- Start `StringConcatSpeed` and let it run till it says `Press return to continue...`.